

API Documentation

Table of Contents

1. [Pet Class API](#)
 2. [Game Class API](#)
 3. [UIController Class API](#)
 4. [StorageManager Class API](#)
 5. [AICompanion Class API](#)
 6. [Constants Reference](#)
 7. [Type Definitions](#)
-

Pet Class API

Constructor

```
/**
 * Create a new pet instance
 * @param {string} name - Pet's name (2-20 characters)
 * @param {string} type - Pet type ('dog', 'cat', 'rabbit', 'bird')
 */
const pet = new Pet('Buddy', 'dog');
```

Properties

Stats (0-100):

- **hunger** - How hungry the pet is (higher = hungrier)
- **energy** - Pet's energy level
- **happiness** - Pet's happiness level
- **cleanliness** - Pet's hygiene level
- **health** - Pet's overall health

Information:

- **name** - Pet's name
- **type** - Pet type
- **createdAt** - Timestamp of pet creation
- **isSick** - Boolean indicating sickness

Tracking:

- **actionsPerformed** - Total actions done
- **feedCount** - Times fed
- **playCount** - Times played with
- **healthChecks** - Health check count

Methods

getEmotion()

```
const emotion = pet.getEmotion();  
// Returns: { emoji: '😊', label: 'Happy' }
```

Logic:

- Returns emotion based on current stats
- Checks health first (sickness = sick emoji)
- Then energy, hunger, happiness in order
- Falls back to neutral

Possible Returns:

- { emoji: '🤒', label: 'Sick' }
- { emoji: '😴', label: 'Sleepy' }
- { emoji: '😡', label: 'Angry' }
- { emoji: '😞', label: 'Sad' }
- { emoji: '😊', label: 'Happy' }

getWellnessState()

```
const wellness = pet.getWellnessState();  
// Returns: { threshold: 40, emoji: '🤒' }
```

States:

- **sick**: average health < 40
- **weak**: average health < 50
- **normal**: average health < 70
- **healthy**: average health < 90
- **excellent**: average health >= 90

updateStats()

```
const changes = pet.updateStats();  
// Returns: { decreased: [...], increased: [...], warnings: [...] }
```

Behavior:

- Degrades all stats by configured amounts
- Applies sickness damage if sick

- Checks for new sickness onset
- Generates warnings for critical stats

Returned Warnings:

- **'hungry'** - Hunger > 85%
- **'tired'** - Energy < 20%
- **'sad'** - Happiness < 30%
- **'dirty'** - Cleanliness < 30%
- **'sick'** - Pet is sick

performAction(action)

```
const result = pet.performAction('feed');  
// Returns: { success: true, message: 'Successfully performed feed!' }
```

Parameters:

- **'feed'** - Reduce hunger, improve health
- **'play'** - Increase happiness, drain energy
- **'rest'** - Restore energy and health
- **'clean'** - Improve cleanliness, prevent illness
- **'healthCheck'** - Monitor/treat health
- **'buyToy'** - Increase happiness

Return Format:

```
{  
  success: boolean,  
  message: string  
}
```

makeSick() / treatSickness()

```
pet.makeSick();           // Force sickness  
const treated = pet.treatSickness(); // Treat if possible
```

timeSinceAction(action)

```
const minutes = pet.timeSinceAction('feed');  
// Returns: number of minutes since action
```

getStats()

```
const stats = pet.getStats();  
// Returns: { hunger: 75, energy: 50, happiness: 80, cleanliness: 60, health: 85 }
```

getStatus()

```
const status = pet.getStatus();  
// Returns: formatted string with pet info
```

toJSON() / fromJSON()

```
// Serialize for storage  
const json = pet.toJSON();  
  
// Restore from storage  
const restoredPet = Pet.fromJSON(json);
```

Game Class API

Constructor

```
const game = new Game();
```

Properties

Game State:

- `pet` - Pet instance
- `isRunning` - Game active state
- `isPaused` - Game paused state

Financial:

- `currentBudget` - Available funds
- `initialBudget` - Starting funds
- `totalSpent` - Total expenses
- `difficulty` - 'easy', 'medium', or 'hard'
- `expenseMultiplier` - Cost multiplier

Session:

- `sessionStartTime` - Game start timestamp
- `updateInterval` - Game loop interval

Collections:

- `unlockedAchievements` - Array of achievements
- `storage` - StorageManager instance

Methods

`initializeGame(name, type, budget, difficulty)`

```
const success = game.initializeGame('Buddy', 'dog', 500, 'medium');  
// Returns: boolean
```

Validation:

- Pet name: 2-20 characters
- Pet type: Valid type from PET_TYPES
- Budget: 100-2000
- Difficulty: 'easy', 'medium', or 'hard'

`performAction(action)`

```
const result = game.performAction('feed');
```

Return Format:

```
{  
  success: boolean,  
  message: string,  
  affordability: boolean,    // Can afford action?  
  warning: string           // Optional warning  
}
```

Costs (adjusted by difficulty):

- feed: \$5
- play: \$3
- rest: \$0
- clean: \$4
- healthCheck: \$10
- buyToy: \$8

`update()`

```
const report = game.update();
```

Return Format:

```
{
  petUpdates: {...},
  achievementUnlocks: [...],
  warningsTriggered: [...],
  gameOverReason: null or string
}
```

Game Over Reasons:

- 'noBudget' - Budget reached \$0
- 'petDied' - Health reached 0%
- 'petLost' - Happiness reached 0%

checkAchievements()

```
const newAchievements = game.checkAchievements();
// Returns: array of newly unlocked achievements
```

endGame(reason)

```
const result = game.endGame('noBudget');
```

Return Format:

```
{
  reason: string,
  message: string,
  stats: {
    petName: string,
    daysCared: number,
    actionsPerformed: number,
    totalSpent: number,
    budgetRemaining: number,
    achievementsUnlocked: number
  }
}
```

getGameStats()

```
const stats = game.getGameStats();
```

Return Structure:

```
{
  petStats: {...},
  petInfo: {...},
  financials: {
    initialBudget: number,
    currentBudget: number,
    totalSpent: number,
    budgetUsedPercentage: string,
    difficulty: string
  },
  progress: {
    actionsPerformed: number,
    feedCount: number,
    playCount: number,
    daysCared: number,
    toysPurchased: number,
    healthChecks: number,
    illnessCount: number
  },
  achievements: {
    unlocked: array,
    count: number,
    totalAvailable: number
  }
}
```

getWeeklyCosts()

```
const costs = game.getWeeklyCosts();
```

Return Format:

```
{
  totalWeeklyCost: number,
  costByAction: {
    feed: number,
    play: number,
    clean: number,
    healthCheck: number,
  }
}
```

```
    buyToy: number
  },
  averageDailyCost: string,
  expenseCount: number
}
```

`saveGame()` / `loadGame()` / `reset()`

```
game.saveGame();    // Save to localStorage
game.loadGame();    // Load from localStorage
game.reset();       // Clear all data
```

UIController Class API

Constructor

```
const ui = new UIController(game);
```

Methods

`_performAction(action)`

```
ui._performAction('feed');
```

Process:

1. Validates game is running
2. Disables button temporarily
3. Calls `game.performAction()`
4. Shows feedback modal
5. Animates button
6. Updates all UI

`_updateAllUI()`

```
ui._updateAllUI();
```

Updates:

- Pet display

- Stat bars
- Financial information
- Achievements list

`_updateStatBars()`

```
ui._updateStatBars();
```

Updates all stat bar widths and values.

`_updateFinancials()`

```
ui._updateFinancials();
```

Updates budget, spent, and weekly cost displays.

`_showFeedback(result, type)`

```
ui._showFeedback({ action: 'feed', message: 'Fed!' }, 'success');
```

Types:

- 'success' - Green feedback
- 'error' - Red feedback

`_showWarning(message)`

```
ui._showWarning('Budget running low!');
```

Shows warning modal with message.

`_openModal(modal) / _closeModal(event, modal)`

```
ui._openModal(this.elements.helpModal);  
ui._closeModal(event);
```

`_handleKeyboard(event)`

```
// Automatically called on keydown  
// Supports shortcuts:  
// F, P, R, C, H, T, ?, Ctrl+S, Esc
```

`_saveGame()` / `_resetGame()`

```
ui._saveGame();    // Save game  
ui._resetGame();   // Start over
```

StorageManager Class API

Constructor

```
const storage = new StorageManager();
```

Methods

`saveGame(gameData)` / `loadGame()`

```
storage.saveGame(gameData);  
const data = storage.loadGame();
```

`savePet(pet)` / `loadPet()`

```
storage.savePet(pet);  
const pet = storage.loadPet();
```

`saveExpense(expense)` / `loadExpenses()`

```
storage.saveExpense({ action: 'feed', cost: 5 });  
const expenses = storage.loadExpenses();
```

Expense Format:

```
{  
  timestamp: number,
```

```
    action: string,  
    cost: number,  
    budgetRemaining: number  
}
```

saveAchievements(achievements) / loadAchievements()

```
storage.saveAchievements(achievements);  
const achievements = storage.loadAchievements();
```

exportData() / importData(jsonString)

```
const json = storage.exportData();    // Export all data  
storage.importData(json);             // Import all data
```

hasSavedGame()

```
if (storage.hasSavedGame()) {  
    // Load game  
}
```

deleteGame()

```
storage.deleteGame(); // Clear all storage
```

getStorageStats()

```
const stats = storage.getStorageStats();  
// Returns: { totalSize, entries, breakdown }
```

clearOldData(daysToKeep)

```
const removed = storage.clearOldData(30);  
// Removes data older than 30 days
```

Constructor

```
const ai = new AICompanion(game);  
// Loads saved Groq key from localStorage automatically
```

Methods

hasApiKey()

```
ai.hasApiKey(); // Returns: boolean
```

Returns `true` if a Groq API key has been saved (key length > 10).

setApiKey(key)

```
ai.setApiKey('gsk_...');  
// Saves key to this._apiKey and localStorage('petpal_groq_key')
```

chatStream(userMessage, onToken, onComplete, onError)

```
ai.chatStream(  
  'How is my pet feeling?',  
  (token) => console.log(token),      // streamed token  
  (fullText) => console.log(fullText), // complete response  
  (err) => console.error(err)         // error handler  
);
```

Streams a response from the Groq API. Requires `hasApiKey()` to return `true`. The system prompt is built from the current pet stats, mood, and extracted memory facts.

Constants Reference

GAME_CONFIG

```
GAME_CONFIG = {  
  UPDATE_INTERVAL: 5000,           // 5 seconds  
  STAT_DEGRADATION: {  
    hunger: 1.5,  
    energy: 1,  
    happiness: 0.8,  
    cleanliness: 1.2,
```

```
    health: 0.3
  },
  DIFFICULTY_MULTIPLIERS: {
    easy: 0.7,
    medium: 1.0,
    hard: 1.5
  }
}
```

PET_TYPES

```
PET_TYPES = {
  dog: { emoji: '🐶', displayName: 'Dog', ... },
  cat: { emoji: '🐱', displayName: 'Cat', ... },
  // ... others
}
```

ACTION_COSTS

```
ACTION_COSTS = {
  feed: 5,
  play: 3,
  rest: 0,
  clean: 4,
  healthCheck: 10,
  buyToy: 8
}
```

ACTION_EFFECTS

```
ACTION_EFFECTS = {
  feed: { hunger: -30, happiness: 2, ... },
  play: { happiness: 15, energy: -10, ... },
  // ... others
}
```

BADGES

12 unlockable badges defined in `constants.js`:

```
BADGES = [
  { id: 'first_steps', name: 'First Steps', desc: 'Perform your first care action.' },
  { id: 'feeding_pro', name: 'Feeding Pro', desc: 'Feed your pet 10 times.'
```

```

},
{ id: 'playmate',      name: 'Playmate',      desc: 'Play with your pet 20
times.' },
{ id: 'night_owl',     name: 'Night Owl',      desc: 'Put your pet to sleep 10
times.' },
{ id: 'clean_freak',   name: 'Clean Freak',     desc: 'Clean your pet 15 times.'
},
{ id: 'vet_regular',   name: 'Vet Regular',     desc: 'Visit the vet 5 times.'
},
{ id: 'chore_star',    name: 'Chore Star',      desc: 'Complete 10 chores.' },
{ id: 'money_maker',   name: 'Money Maker',     desc: 'Earn 200 coins from
chores.' },
{ id: 'big_saver',     name: 'Big Saver',       desc: 'Earn 500 coins total.' },
{ id: 'level_five',    name: 'Growing Up',     desc: 'Reach Level 5.' },
{ id: 'level_ten',     name: 'Pet Master',      desc: 'Reach Level 10.' },
{ id: 'all_well',      name: 'Perfect Care',    desc: 'Keep all stats above 70
simultaneously.' }
]

```

TRICKS

9 tricks unlocked by pet level:

```

TRICKS = [
{ id: 'sit',           name: 'Sit',             level: 2 },
{ id: 'shake',         name: 'Shake Hands',     level: 3 },
{ id: 'roll_over',     name: 'Roll Over',       level: 4 },
{ id: 'dance',         name: 'Dance',           level: 5 },
{ id: 'spin',          name: 'Spin',            level: 6 },
{ id: 'jump',          name: 'High Jump',       level: 7 },
{ id: 'speak',         name: 'Speak',           level: 8 },
{ id: 'handstand',     name: 'Handstand',       level: 9 },
{ id: 'perform',       name: 'Grand Performance', level: 10 }
]

```

CHORES

6 chores that earn coins (cooldown in ms):

```

CHORES = [
{ id: 'dishes',        name: 'Wash the Dishes',  reward: 20, cooldown: 5min },
{ id: 'room',          name: 'Clean Your Room',  reward: 25, cooldown: 15min },
{ id: 'plants',        name: 'Water the Plants',  reward: 15, cooldown: 10min },
{ id: 'trash',         name: 'Take Out the Trash', reward: 15, cooldown: 10min },
{ id: 'dog_walk',      name: "Walk Neighbor's Dog", reward: 35, cooldown: 30min },
{ id: 'groceries',     name: 'Help with Groceries', reward: 30, cooldown: 20min }
]

```

Type Definitions

Emotion

```
{
  emoji: string,      // Unicode emoji
  label: string        // 'Happy', 'Sad', etc.
}
```

Pet Stats

```
{
  hunger: number,      // 0-100
  energy: number,      // 0-100
  happiness: number,   // 0-100
  cleanliness: number, // 0-100
  health: number       // 0-100
}
```

Action Result

```
{
  success: boolean,
  message: string,
  affordability?: boolean,
  warning?: string
}
```

Achievement

```
{
  id: string,
  title: string,
  description: string,
  icon: string,
  condition: function
}
```

Game Stats

```
{
  petStats: Object,
  petInfo: Object,
```

```
financials: Object,  
progress: Object,  
sessionInfo: Object,  
achievements: Object  
}
```

Last Updated: May 2026